Machine Learning

Course Summary

1. Linear Regression

Overview:

The first week of the Machine Learning course focuses on Linear Regression, a foundational technique in predictive modeling and data analysis. This week introduces you to the methods used to quantify relationships between variables, visualize complex data, and implement linear regression to make reliable forecasts. This week covers essential concepts related to linear relationships in data, reinforced through practical applications and hands-on projects aimed at enhancing the understanding of linear relationships in data.

Key Topics and Concepts:

- **Business Problems definition:** This section highlights the significant role of linear regression in addressing various business challenges. It involves the definition of specific problems that linear regression models aim to solve, focusing on the articulation of clear objectives for the analysis. The focus is on establishing quantitative relationships between independent (predictor) variables and a dependent (outcome) variable. Additionally, the importance of crafting a clear and measurable problem statement is discussed, ensuring that it effectively guides the analytical process and aligns with overall business goals.

  The discussion includes relevant examples that demonstrate how organizations utilize linear regression analysis to streamline operations and optimize marketing efforts, ultimately preparing participants for focused analyses and actionable outcomes.

- **Concept of Correlation among variables:** Emphasizing the importance of data visualization, this week covers techniques that allow for effective examination of relationships and correlations between variables. Scatter plots and correlation

matrices are employed to graphically represent data points and trends. This week focuses on identifying patterns that could suggest direct, inverse, or no correlation between predictor and outcome variables. This critical analysis helps understand the relationship dynamics intuitively.

- **Pearson's Correlation Coefficient:** This section introduces Pearson's correlation coefficient as a key statistical measure that quantifies the strength and direction of the linear relationship between two variables. It covers both the calculation and interpretation of the coefficient, which ranges from -1 (indicating a perfect negative correlation) to +1 (indicating a perfect positive correlation). This understanding of Pearson's correlation provides the groundwork for determining which relationships are worth modeling with linear regression.
- **Simple Linear Regression:** In this core component, the concept of simple linear regression was explored in depth. It helps to analyze how to model the relationship between a single independent variable and a dependent variable using a straightforward linear equation, which represents a best-fit line through the data points. It also covers examples on how to derive predictions based on the established relationships, enhancing their ability
- **Coefficient Interpretation for Linear Regression:** This section delves deeper into the significance of regression coefficients derived from their models. This topic clarified the roles of the intercept and slope in the linear equation, elucidating what these coefficients indicate about the relationship between predictor variables and the outcome variable. Interpreting the coefficients helps in understanding the implications of changes in independent variables on predicted outcomes, such as how increasing advertising spend might affect sales revenue.
- **Best-fit Line (Concept and Computation):** This section focuses on the best-fit line, or regression line, which signifies the optimal association between independent and dependent variables in a regression model. It explores the least squares method, which minimizes the sum of squared distances between observed data points and model predictions, highlighting the line's significance in making accurate forecasts and analyzing trends. Additionally, it covers a practical walkthrough to compute the best-fit line using statistical formulations, understanding how to determine the slope (m) and intercept (b). This mastery

not only teaches the technical skills to perform calculations but also deepens the appreciation of the mathematical principles underlying regression analysis.

- **Multiple Linear Regression:** This topic introduces multiple linear regression as an advanced extension of simple linear regression that allows for the incorporation of multiple predictor variables simultaneously. It explores how this enhanced model complexity provides a richer understanding of complex relationships within data.
- **Categorical Variables in Regression:** This section uncovers strategies for incorporating categorical variables into regression models, emphasizing the need to convert these variables into a numerical format for analysis. Techniques such as label encoding and dummy variables are discussed, illustrating how categorical data can effectively be integrated into linear regression without distorting the model's interpretability.
- **Evaluation Metrics for Regression:** In this section, various metrics that evaluate the performance of regression models are introduced. Metrics like R-squared, which indicates the proportion of variance explained by the model, along with Mean Absolute Error (MAE) and Mean Squared Error (MSE), are defined and discussed. Understanding these evaluation metrics provides the tools necessary to assess model effectiveness and make required adjustments to improve performance.

Metric	Summary	Formula
Mean Absolute Error (MAE)	Tells us the average error in predictions, without worrying about whether the predictions were too high or too low.	$MAE = \frac{1}{n}\sum_{i=1}^{n}\left\|y_i - \hat{y}_i\right\|$
Mean Absolute Percentage Error (MAPE)	Shows how accurate predictions are as a percentage, making it easy to understand how big the errors are compared to actual values.	$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left\|\frac{y_i - \hat{y}_i}{y_i}\right\| * 100$

Root Mean Squared Error (RMSE)	Gives a number that shows the average size of the errors, giving more weight to larger mistakes.	$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$
R-squared (R^2)	Shows how well our model explains the data; a higher number means a better fit.	$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$
Adjusted R-squared	Like R-squared, but accounts for the number of predictors, making it a better measure to compare models and prevent overfitting.	$Adjusted\ R^2 = 1 - \frac{(1-R^2)*(n-1)}{(n-k-1)}$

- **Hands-On Application of Linear Regression:** This hands-on section focuses on solving a business problem using Linear Regression. It includes defining a real-world case study, loading and preparing the dataset, and performing exploratory data analysis (EDA) in Python. Predictive models are built using simple and multiple linear regression, with both numerical and categorical variables. The practical application of these concepts helps to evaluate model performance, make data-driven predictions, and derive actionable insights, reinforcing the understanding of linear regression in a real-world context.
- **Key Functions:**
  - train_test_split(X, y, test_size, random_state): Splits the independent variables X and dependent variable y into training and testing sets.
  - pd.get_dummies(X, columns, drop_first): Converts categorical variables in a DataFrame X into dummy/one-hot encoded variables.
  - LinearRegression(): Creates an instance of the linear regression model from scikit-learn.
  - lin_reg.fit(X_train, y_train): Fits the linear regression model to the training data, where X_train are the independent variables and y_train is the dependent variable.

- lin_reg.predict(X_test): Predicts the dependent variable values for the test data X_test using the fitted linear regression model.
- lin_reg.coef_: Returns the coefficients (slopes) of the independent variables in the fitted linear regression model.
- lin_reg.intercept_: Returns the intercept (bias) of the fitted linear regression model.

2. Decision Tree

Overview

The second week of the Machine Learning course focused on Decision Trees, a robust algorithm used for both classification and regression tasks. This week provided an in-depth understanding of how Decision Trees function, their applications in solving real-world problems, and the evaluation metrics necessary for assessing their performance. Through a blend of theoretical principles and hands-on activities, this week focuses on leveraging Decision Trees effectively in business analytics.

Key Topics and Concepts

- **Decision Trees - Business Problem Space:** This section explores how Decision Trees can address specific business problems, such as classifying customers or products and predicting outcomes based on various inputs. Real-world examples highlight the utility of Decision Trees in making consequential and informed business decisions.
- **Decision Trees - Overview:** Decision Trees provide a powerful visual and analytical framework for understanding how decisions are made based on input data. This section defines the structural components of Decision Trees, such as nodes (decision points), branches (possible outcomes), and leaves (final predictions), and helps grasp how trees partition data into meaningful subsets. Emphasis is placed on the visual representation of these structures, which helps illustrate the relationships between features and outcomes. This not only aids in interpreting the decision-making process but also enables business analysts to effectively communicate data-driven insights through intuitive, tree-based diagrams.

- **Impurity Measures:** This discussion covers impurity measures, introduces key concepts like Gini impurity and entropy, which are vital for assessing the quality of splits within the Decision Tree. It emphasizes how these metrics guide the model in selecting the best attributes for splitting the data, ensuring that each division increases the homogeneity of the resulting subsets. By minimizing impurity, the Decision Tree becomes more effective at making accurate predictions, as it systematically organizes data based on the most informative features.
- **Decision Trees - Splitting Mechanism:** This topic explores in detail the mechanism by which Decision Trees determine the optimal way to split the data at each node using impurity measures. It highlights how these splits are crucial for maximizing the information gain or minimizing the impurity of the resulting subsets. By grasping the splitting mechanism, it focuses on understanding how choices made at each node directly impact the accuracy and interpretability of the model, ultimately influencing the outcomes of their analyses.

  The steps involved in the splitting mechanism are as follows:

  - **Examine All Features:** Look at all the available features (or predictor variables) in the dataset to determine which one could provide the best split.
  - **Calculate Impurity for Each Feature:** For each feature, compute the impurity for different potential split points (thresholds). This involves calculating the impurity for the left and right branches created by the split.
  - **Evaluate Information Gain:** For each feature and split point, measure the information gain (or reduction in impurity) resulting from that split. Higher information gain indicates a better split.
  - **Select the Best Split:** Choose the feature and split point that results in the highest information gain (or lowest impurity). This will be the split for the node.
  - **Create Child Nodes:** Split the data into two subsets (child nodes) based on the chosen split. Each child node now contains a portion of the data that corresponds to the left or right side of the split.

- **Repeat the Process:** For each child node, repeat the above steps (choose a split criterion, examine features, calculate impurity, evaluate information gain, and create further splits) until a stopping criterion is met.
- By following these steps, Decision Trees can effectively partition data, enhancing the model's performance and providing clear, interpretable paths for making predictions.

- **Impurity Computation (Gini Impurity and Entropy):** This section helps develop a solid understanding of impurity measures by engaging with the computation of both Gini impurity and entropy, two key metrics used in Decision Trees to evaluate the quality of splits. Through hands-on examples, it covers the practical implementation of calculating Gini impurity to quantitatively assess how well a split increases the homogeneity of data subsets. Alongside this, this section also explores entropy as a measure of uncertainty within a dataset, learning its formula and its role in guiding data partitioning based on class label distributions. Together, these computations deepen the ability to make informed, data-driven decisions during tree construction.

Impurity Measure	Summary	Formula
Gini Impurity	Measures how often a randomly chosen element would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset. Lower values indicate purer subsets.	$Gini\ Impurity = \sum_{i=1}^{k} p_i^2$
Entropy	Quantifies the uncertainty or disorder in a set of classes. A higher value means more impurity, while a lower value indicates that the set is more uniform and predictable.	$Entropy = -\sum_{i=1}^{k} p_i log_2(p_i)$

- **Classification - Evaluation Metrics:** Evaluating the performance of Decision Trees is crucial for assessing their effectiveness in classification tasks. This section introduces a variety of evaluation metrics, including accuracy, precision, recall, and the F1-score, which provide insights into the model's performance. Understanding these metrics helps us to assess how well their models performed on new/unseen data and enables us to identify necessary adjustments for improving the model's performance.

  The major evaluation metrics for classification problems are listed below:

Metric	Summary	Formula
Accuracy	Measures the overall correctness of the model by calculating the proportion of true results (both true positives and true negatives) among the total number of cases examined.	$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
Precision	Indicates how many of the predicted positive instances are actually positive, helping to assess the exactness of the positive predictions.	$Precision = \frac{TP}{TP+FP}$
Recall	Measures how well the model identifies actual positive instances, indicating the model's ability to capture relevant cases.	$Recall = \frac{TP}{TP+FN}$
F1-Score	Combines precision and recall into a single metric, providing a balance between the two, particularly useful when the class distribution is imbalanced.	$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$

- **Model Complexity vs Performance:** This topic examines the trade-offs inherent in Decision Trees regarding model complexity and performance. It delves deeper into the risks of overfitting, where a model becomes too complex and captures noise in the data rather than the intended relationships. By emphasizing the importance of finding a balance, this section highlights the importance of managing complexity while maintaining model accuracy.
- **Decision Trees - Pruning:** To enhance model generalization and prevent overfitting, we cover pruning methods that simplify Decision Trees by eliminating unnecessary branches.
    - **Pre-pruning:** It explores early stopping strategies, such as limiting tree depth or setting a minimum number of samples required to split a node. These techniques help control tree growth during the training process, ensuring that the model doesn't become overly complex.

      The most common parameters to experiment with while pre-pruning a decision tree are listed below.

Parameter	Summary
Max Depth	Limits how deep the tree can grow, preventing it from creating overly complex structures and reducing overfitting.
Min Samples Leaf	Sets the minimum number of samples that must be present in a leaf node. This prevents nodes from being created that capture very few data points, promoting robustness.
Min Samples Split	Specifies the minimum number of samples required to split an internal node. If a node has fewer samples, it won't split further.
Max Leaf Nodes	Limits the total number of leaf nodes in the tree, which can help in simplifying the model and avoiding overfitting.

Max Features	Restricts the number of features to consider when looking for the best split, introducing randomness that can help generalization and reduce overfitting.
Criterion	Defines the function used to measure the quality of a split. Common options include "gini" for Gini impurity and "entropy" for information gain.
Class Weight	Used to assign different weights to classes, which is particularly useful in imbalanced datasets to give more importance to minority classes.

    - **Post-pruning (Cost Complexity Pruning):** This part examines cost complexity pruning, a post-training technique that prunes the tree by balancing model accuracy with simplicity. This method involves penalizing overly large trees through a complexity parameter, retaining only those branches that contribute meaningfully to predictive performance.
- **Decision Trees for Regression:** Expanding the knowledge, this section introduces the use of Decision Trees for regression tasks, illustrating how they can predict continuous outcomes rather than categorizing discrete classes. It compares the methodologies of classification and regression Decision Trees, which provided insight into the versatility of tree-based approaches across varied analytical contexts.

  Some major differences between classification and regression trees are as follows:

    - Classification trees predict categorical outcomes, assigning data points to discrete classes based on features. In contrast, regression trees aim to predict continuous values.
    - When it comes to selecting the final value, classification trees determine the most common class label among the instances within a leaf node, effectively using majority voting. On the other hand, regression trees

compute the final prediction by averaging the continuous target values of the instances in the leaf node.

- The impurity measures used for both these trees are also different: classification trees employ Gini impurity or entropy, while regression trees utilize metrics like mean squared error (MSE) or mean absolute error (MAE).

- **Hands-On Application of Decision Tree:** This hands-on segment of Week 2 focuses on applying Decision Tree concepts to a real-world business problem. It begins with defining the problem and performing exploratory data analysis (EDA) using descriptive statistics and visualizations. The data is then cleaned and prepared for modeling. A basic Decision Tree model is built and visualized to understand model fitting and interpretation. Pre-pruning techniques are applied to control model complexity and improve performance, followed by post-pruning methods for further refinement. Finally, multiple Decision Tree models are compared using evaluation metrics to assess and improve predictive performance.
- **Key Functions:**
  - train_test_split(arrays, test_size, train_size, random_state, shuffle, stratify): Splits arrays or matrices into random train and test subsets.
  - DecisionTreeClassifier(criterion, splitter, max_depth, min_samples_split, min_samples_leaf, min_weight_fraction_leaf, max_features, random_state, max_leaf_nodes, min_impurity_decrease, class_weight, ccp_alpha): Creates an instance for the Decision Tree Classifier.
  - confusion_matrix(y_true, y_pred, labels, sample_weight, normalize): Computes a confusion matrix to evaluate the accuracy of a classification.
  - accuracy_score(y_true, y_pred, normalize, sample_weight): Computes the accuracy classification score.
  - recall_score(y_true, y_pred, labels, pos_label, average, sample_weight, zero_division): Computes the recall.
  - precision_score(y_true, y_pred, labels, pos_label, average, sample_weight, zero_division): Compute the precision.
  - f1_score(y_true, y_pred, labels, pos_label, average, sample_weight, zero_division): Compute the F1 score, also known as balanced F-score or F-measure.

- tree.plot_tree(decision_tree, max_depth, feature_names, class_names, label, filled, impurity, node_ids, proportion, rounded, precision, ax, fontsize): Plots a decision tree.
- tree.export_text(decision_tree, feature_names, max_depth, spacing, decimals, show_weights): Build a text report showing the rules of a decision tree.
- DecisionTreeClassifier.cost_complexity_pruning_path(X,y,sample_weight) : Compute the pruning path during Minimal Cost-Complexity Pruning.
- DecisionTreeClassifier.predict(X): Predict class for X.
- DecisionTreeClassifier.predict_proba(X): Predict class probabilities for X.

3. Clustering

Overview:

In the third week of the Machine Learning course, the focus shifted to unsupervised learning techniques, with an emphasis on clustering, particularly K-Means Clustering. This unsupervised learning method is essential for grouping data into distinct clusters based on similarity, enabling businesses to uncover patterns within their datasets. This week delved deeper into various clustering methodologies, visualizing data relationships effectively, and conducting hands-on activities to gain practical insights into customer and product segmentation. The skills acquired during this week help us to leverage clustering in our analytical practices.

Key Topics and Concepts

- **Business Problem Space - Clustering:** This section covers the use of clustering to address business problems such as identifying customer segments, improving marketing strategies, and optimizing product offerings. It includes defining specific scenarios to demonstrate how clustering can be applied in real-world contexts. The focus is on understanding the value of clustering in generating insights and supporting strategic decisions.
- **Distance Metrics for Assessing Similarities:** This section emphasizes the significance of visualizing relationships in data to better understand clusters. It explores distance metrics, particularly Euclidean distance and Manhattan distance, as tools for quantifying how similar or dissimilar data points are to one

another. Understanding these metrics allows us to establish the foundational basis for effective clustering analyses and helps to see how clustering relies on the geometric relationships of data points.

The most common distance metrics used are as follows:

Distance Metric	Summary	Formula
Euclidean Distance	Measures the straight-line distance between two points in Euclidean space, representing the shortest path. It is the most common distance metric used for clustering.	$d(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
Manhattan Distance	Calculates the distance between two points by summing the absolute differences of their coordinates, representing a path along axes at right angles (like navigating a city grid).	$d(x,y) = \sum_{i=1}^{n} \lvert x_i - y_i \rvert$

This segment also discusses the importance of scaling variables in distance-based methods. It emphasizes the Z-score scaling technique, which is widely utilized for feature scaling in K-Means Clustering.

$$Z\_score = \frac{x-\mu}{\sigma}$$

- **Introduction to Clustering:** This topic provides a comprehensive introduction to clustering techniques, focusing on two primary approaches: connectivity-based

and centroid-based clustering. It explores how connectivity-based methods form clusters by linking closely related data points based on distance or density, while centroid-based methods define clusters around central points or centroids. The session emphasizes the importance of using a mathematical model to analyze and interpret the relationships and patterns revealed within the data.

- **K-Means Clustering:** A deep dive into the K-Means algorithm reveals it as one of the most popular clustering methods. This section highlights how the algorithm partitions data into K distinct groups through an iterative process that includes initializing centroids, assigning data points to clusters based on proximity, and updating centroids until convergence is achieved. This exploration helps us grasp how K-Means effectively segments data to reveal underlying structures.

  The steps involved in the K-Means Clustering algorithm are given below:

  - **Initialize Centroids:** Randomly select K data points from the dataset to serve as the initial centroids (the centers of the clusters).
  - **Assign Clusters:** For each data point in the dataset, calculate the distance to each centroid and assign the data point to the nearest centroid's cluster.
  - **Update Centroids:** After all data points have been assigned to clusters, recalculate the centroid of each cluster by computing the mean of all data points in that cluster.
  - **Repeat:** Go back to the assignment step and repeat the process of assigning clusters and updating centroids until the centroids no longer change significantly or the assignments of data points to clusters stabilize.
  - **Stopping Criteria:** The K-Means algorithm stops iterating when one or more of the following criteria are met:
    - Centroid Stability: The centroids do not change significantly between iterations, indicating that the clusters are stable and the algorithm has converged.
    - No Change in Assignments: The data points remain in the same clusters across iterations, meaning assignments are stable.

- Maximum Iterations Reached: A predefined limit on the number of iterations has been reached. This prevents the algorithm from running indefinitely.

These criteria ensure that the algorithm terminates in a reasonable time frame, producing a clustering result that reflects the underlying data structure.

- **Selecting optimal number of clusters:** The importance of determining the optimal number of clusters (K) is discussed in depth, with techniques like the Elbow Method and the Silhouette Score being highlighted. It covers how to apply these methods to assess clustering performance and how to select an appropriate K-value that yields meaningful results and accurately reflects the data's inherent structure.

  Techniques such as the Elbow Method and the Silhouette Score are commonly used to assess clustering performance and identify an appropriate K-value that accurately reflects the data's inherent structure.

  - **Elbow Method:** This method involves plotting the Within-Cluster Sum of Squares (WCSS), which measures the total variation within each cluster.

$$WCSS = \sum_{j=1}^{k} \sum_{x_i \in C_j} (x_i - \overline{x}_j)^2$$

    As K increases, WCSS generally decreases because adding more clusters tends to reduce the distance between points in the same cluster. To use the Elbow Method, you plot WCSS against the number of clusters (K) and look for the "elbow" point/sharp decline where the rate of decrease sharply slows down. This point indicates a balance between the number of clusters and the compactness of the clusters, suggesting a suitable K.

  - **Silhouette Score:** The Silhouette Score evaluates how well-separated the clusters are. It ranges from -1 to +1, where a score close to +1 indicates that the data points are well-clustered and far from other

clusters, while a score near -1 suggests that points might be in the wrong cluster.

$$s(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

Where:

- a(i) is the average distance from the data point i to all other points in the same cluster.
- b(i) is the average distance from the data point i to all points in the nearest cluster.

To interpret the Silhouette Score, a higher score (closer to +1) suggests that the clustering configuration is good, while a low or negative score indicates that the data points might not be clustered correctly.

- **Cluster Profiling:** This section emphasizes the process of cluster profiling, which involves analyzing and interpreting the characteristics of each cluster after segmentation. This profiling enables businesses to understand their distinct customer segments and tailor their strategies accordingly.
- **Need for Dimensionality Reduction:** The necessity of dimensionality reduction techniques is introduced as a means to simplify complex datasets for clustering analysis. Participants discovered that reducing the dimensions of the data can enhance model performance and visualization, making it easier to identify clusters without compromising critical information. This section underscores the importance of dimensionality reduction in improving computational efficiency and interpretability of clustering results.
- **t-SNE Overview:** This section introduces t-SNE, which stands for t-distributed Stochastic Neighbor Embedding. It is a non-linear dimensionality reduction technique for visualizing high-dimensional data by mapping it to lower dimensions, typically 2 or 3. This method effectively visualizes clusters while preserving the local structure of the data, aiding in the identification of latent patterns.

The algorithm computes a distribution that measures pairwise similarities in the original data, then seeks a close lower-dimensional mapping of these similarities by minimizing divergences iteratively. This process yields a visualization that maintains meaningful relationships within the data.

t-SNE helps visually cluster data points based on similarity, but the lower-dimensional features obtained lack interpretable meaning. **Perplexity** is a parameter that affects how t-SNE gauges the local structure of the data. It can be understood as a measure of the number of neighbors each data point considers when determining similarity, thus influencing the formation and density of clusters. The parameter perplexity can be fine-tuned to represent high-dimensional data better while influencing visualization quality by providing insight into the number of neighbors considered when forming clusters.

Visualization examples using different perplexity values show how clusters can vary based on this parameter, enabling optimal settings for clearer outputs. Understanding t-SNE, including the role of perplexity, enhances the capability to analyze and interpret complex datasets effectively.

- **Problem Statement and Data Overview - Clustering Hands-on:** This hands-on section covers the complete clustering workflow, starting with problem definition and feature selection based on the dataset. It includes exploratory data analysis (EDA) and preprocessing steps such as handling missing values and preparing the data for clustering. t-SNE is used to visualize high-dimensional data and identify patterns. The K-Means algorithm is implemented to perform clustering, covering steps from initialization to final cluster formation. The section concludes with cluster profiling to understand the characteristics of each cluster and generate data-driven insights and recommendations.
- **Key Functions:**
    - StandardScaler(): Initializes a StandardScaler object to standardize features by removing the mean and scaling to unit variance.
    - StandardScaler().fit_transform(X): Fits the StandardScaler to the data X and then transforms X by applying the calculated scaling.

- TSNE(n_components, perplexity, n_jobs, random_state): Initializes a t-SNE object with the specified number of components for the reduced dimension, perplexity, number of jobs for parallel processing, and random state for reproducibility.
- TSNE().fit_transform(X): Fits the t-SNE model to the data X and transforms it into the specified number of dimensions.
- KMeans(n_clusters, random_state): Initializes a KMeans object with the specified number of clusters and random state for reproducibility.
- KMeans().fit(X): Computes the KMeans clustering on the data X.
- KMeans().inertia_: Returns the Within-Cluster Sum of Squares (WCSS) for the fitted KMeans model.
- KMeans().labels_: Returns the cluster labels for each data point after fitting the KMeans model.
- silhouette_score(X, labels): Computes the mean Silhouette Coefficient of all samples, which is a measure of how well samples are clustered.